

Coursera - IBM - Deep Learning Phishing Detection

April 22, 2026

1 Overview

For this project I chose to work with the Phishing Websites dataset from UCIML. This dataset describes a set of features that are related to the prediction whether a website is a phishing website or not. Deep learning models will be evaluated on this dataset, together with different dimensionality reduction techniques.

1.1 Project Goal

The main goal of this project is to explore how deep learning methods can be applied to improve on the performance of classical methods. This is done in two contexts

- Dimensionality reduction → An autoencoder is used for dimensionality reduction. This is compared with PCA.
- Prediction modeling → A few-layer neural network is used on the original data and the dimensionality reduced data to see how it performs in relation to several classical methods (Linear regression, Random-Forest, kMeans clustering).

2 Data Description

The source data was gathered and processed by Mohammad, R. and McCluskey, L. in 2012. Their goal was to create a comprehensive dataset that worked well for determining whether a website is a phishing website. Since a lot of web data is unstructured (for example related to the source domain, web lease information and security options), there was no clear consensus on how this should productively be organized. For this they devised a rule based system that converted the complex unstructured data to 31 binary- and ternary-encoded columns. The description of how they did this can be found here, and detailed definitions of the column values can be found here upon downloading the data.

2.1 Technical description

The dataset consists of 11055 rows with 30 features. Of those features, 22 are encoded in binary classes, 8 are encoded in ternary classes. The features are:

- having_ip_address
- url_length
- shortining_service
- having_at_symbol
- double_slash_redirecting
- prefix_suffix
- having_sub_domain
- sslfinal_state
- domain_registration_length
- favicon
- port
- https_token
- request_url

- url_of_anchor
- links_in_tags
- sfh
- submitting_to_email
- abnormal_url
- redirect
- on_mouseover
- rightclick
- popupwindow
- iframe
- age_of_domain
- dnsrecord
- web_traffic
- page_rank
- google_index
- links_pointing_to_page
- statistical_report

The features are all encoded as integers, signifying a class. In the case of binary classes, the possible values are {non-phishing, phishing}. And in the case of ternary classes, the options are {non-phishing, suspicious, phishing}.

The distribution is relatively even between the two output classes, as can be seen in the figure below.

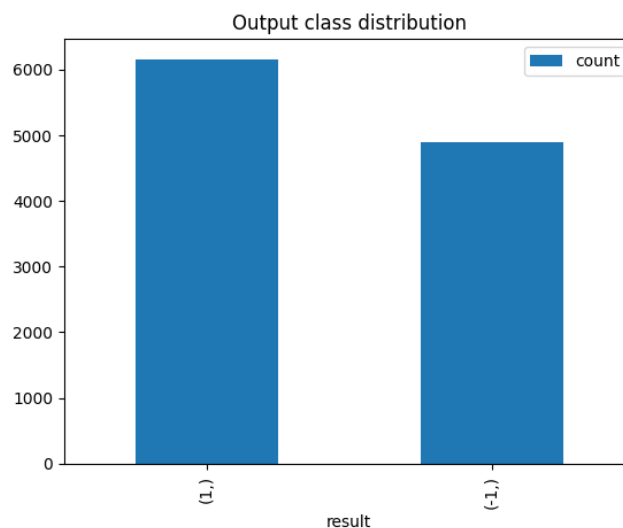


Figure 1: Distribution of output class

2.2 EDA and preprocessing

Since the data was already largely processed due to the rule-based system for binary/ternary encoding, there was not a lot of preprocessing necessary. For example, there were no missing values and the ranges of the columns were clearly defined. However, for machine learning in general and neural networks in particular, there were some important steps that needed to be taken.

First, the encoding of the data was not directly suited for classification. The columns were encoded from the set $\{-1, 1\}$, and needed to be mapped to $\{0, 1\}$. The same transformation was done on the target variable, since the encoding was similarly unsuitable. This was also the case for the ternary columns, where the mapping was $\{-1, 0, 1\} \rightarrow \{0, 1, 2\}$. Then, the data needed to be converted to binary columns only. For this all ternary columns were selected and converted to binary using one hot encoding. This then resulted in a new dataset with 46 features. Below you can see the correlation between each of the variables in a heatmap, together with the highest found correlations with $c > 0.7$. This shows that there is significant reason to suspect dimensionality reduction can be beneficial.

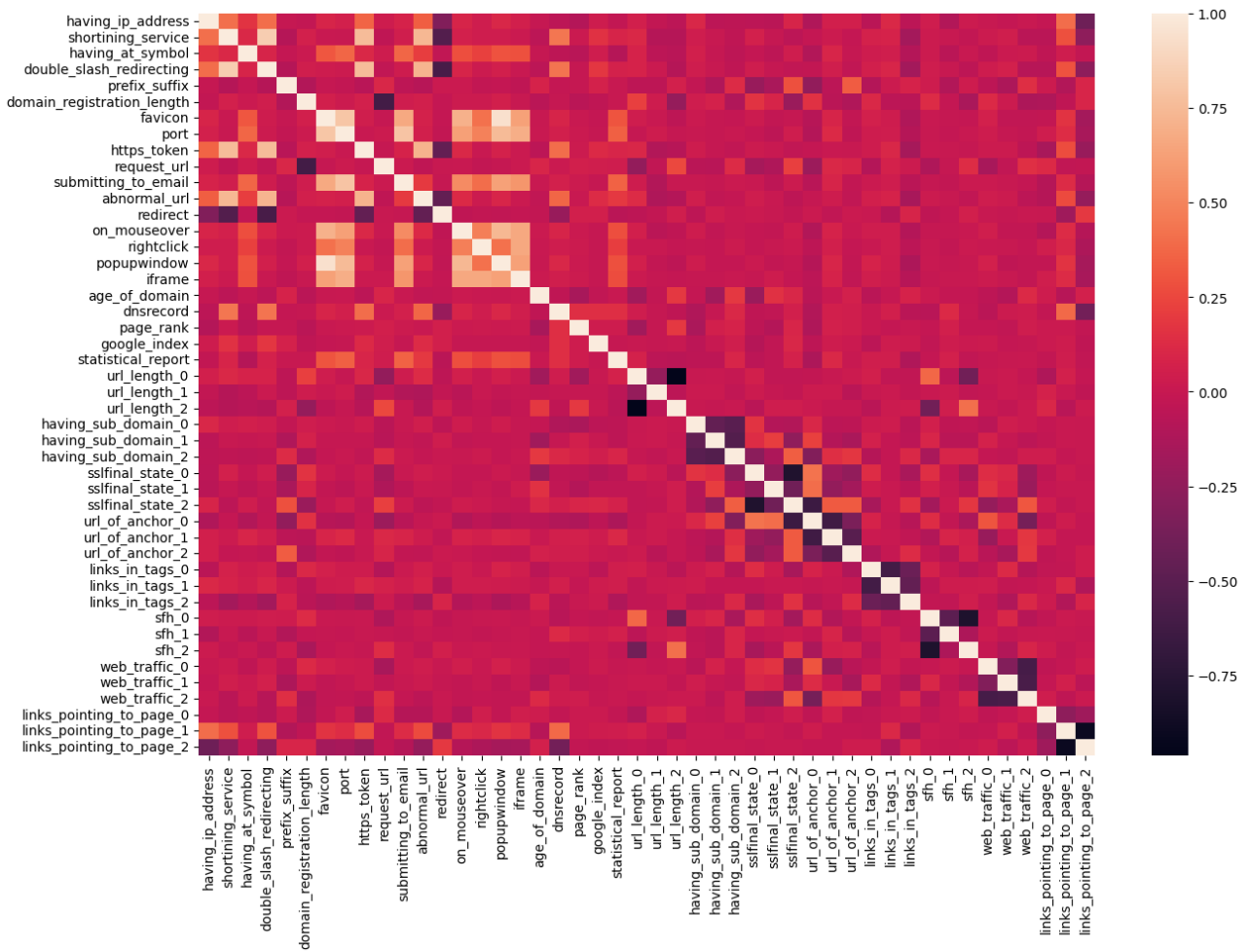


Figure 2: Heatmap of the correlation values for each input feature.

	var_1	var_2	correlation
0	favicon	popupwindow	0.939633
1	shortining_service	double_slash_redirecting	0.842796
2	favicon	port	0.803834
3	port	submitting_to_email	0.799088
4	double_slash_redirecting	https_token	0.760799
5	shortining_service	https_token	0.757838
6	port	popupwindow	0.748517
7	shortining_service	abnormal_url	0.739290
8	on_mouseover	popupwindow	0.733629
9	double_slash_redirecting	abnormal_url	0.723724
10	https_token	abnormal_url	0.716287
11	favicon	on_mouseover	0.706179
12	sslfinal_state_0	sslfinal_state_2	-0.797353
13	sfh_0	sfh_2	-0.806441
14	links_pointing_to_page_1	links_pointing_to_page_2	-0.903074
15	url_length_0	url_length_2	-0.960039

Figure 3: Highest correlation values with $|c| > 0.7$

Then the data was split up into three segments (train, test, validation) with the following proportionality (0.6, 0.2, 0.2). Because of the slight imbalance in the output classes, this was done in a stratified manner.

3 Models

3.1 Dimensionality Reduction

Due to the relatively high correlation values present in the correlation matrix, dimensionality reduction was also explored in the preliminary analysis. For this a quick check was done with PCA and it was determined that 95% of the data variance could be explained with only 27/46 components. These 27 components were selected and put in a reduced dataset used for baseline evaluation.

3.2 Preliminary analysis and baseline

To get a quick baseline measure of prediction performance, a few classical models were trained. These were not meant as optimized comparison models, but as a quick overview of expected unoptimized performance. Below the scores of 3 classical models can be seen after training on the full data set and on the reduced dataset.

	LogisticRegression	RandomForest	SVM
full	0.935776	0.965626	0.954772
reduced	0.934419	0.964722	0.955224

Figure 4: The baseline measures of 3 classical machine learning models over the full- and reduced datasets.

From this we can see that performance is relatively even (between 93% and 97% accuracy) and confirms the predictability of the dataset. Since these figures were obtained on models that used default parameters, it is to be expected that this could still improve slightly with better hyperparameter selection. But for a baseline measure this is a good overview.

3.3 Model description

3.3.1 Neural Network Baseline

The first model for comparison is a single-hidden-layer neural network, with hidden layer size of $\frac{\text{num_features}}{2}$. This model performed quite well with an accuracy score on the validation set of 96%, which is on par with the classical models.

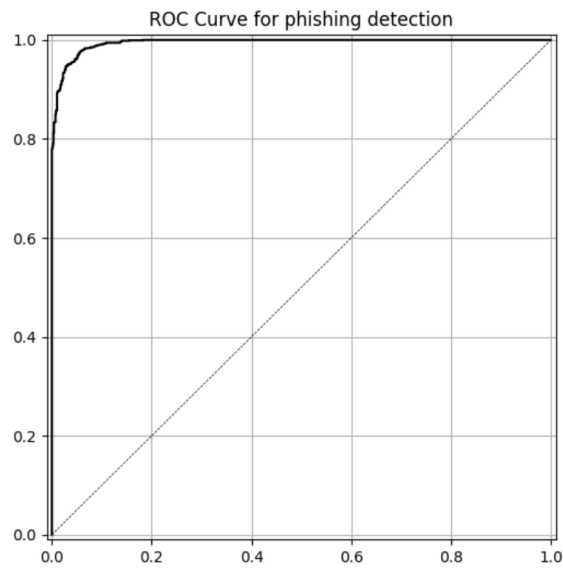


Figure 5: ROC Curve for initial NN. There is good generalisation and coverage.

This simple neural network model performs quite well already. Let's try with a slightly more complicated network to see if we get improved performance.

3.3.2 A more complex Neural Network

The second iteration of the network consisted of 2 hidden layers which became progressively smaller. Each layer reducing the dimensionality of the data by a factor of 2. This model was trained in the same way, and achieved similar performance of 95% accuracy on the validation set. However, from the training history we can clearly see that the validation loss levels off quickly and then starts rising again, indicating overfitting.

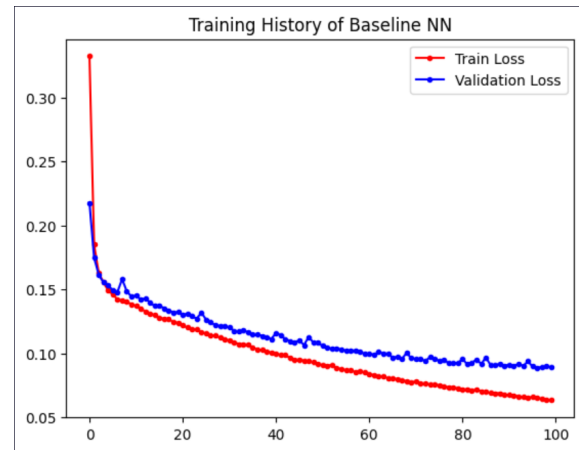


Figure 6: Training history for initial NN. While training loss seems to be improving, the validation is significantly levels off after 100 epochs.

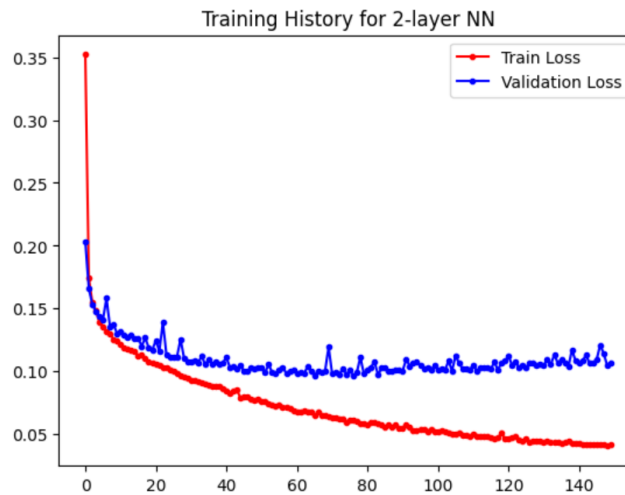


Figure 7: Training history of the 2-hidden-layer NN. The loss quickly levels off and starts rising again. This is a sign that the model is too complex and tends toward overfitting on the training data.

While a slight improvement may be achieved when stopping the training at exactly the right time, this would likely not result in significant progress over multiple trials. Tendency to overfit is very much dependent on the data and as such not a reliable way of selecting models when this tendency is present.

3.3.3 Neural network conclusions

From this we can conclude that the data does not require a complex NN structure to be represented properly. A single hidden layer seems sufficient to capture a useful generalisation and performs well on validation data. Another significant factor in this decision is the size of the model. The base model has 1105 parameters, while the 2-layer model has 4073 parameters. This increase does not justify the slight potential improvement.

Since this neural network is effectively performing a type of dimensionality reduction, it is better served to see if this process can be improved, especially since earlier it was shown that there is a significant degree of redundancy in the data. For this we turn to building a dimensionality reduction preprocessing layer using an autoencoder.

3.4 Autoencoder preprocessing

The relatively naive way of performing PCA for dimensionality reduction seemed to not influence the preliminary baseline measurements. But since there is a reasonable amount of redundancy in the data (explain 95% with about 2/3 of the entire set), a more sophisticated method might work here. For this I chose to implement an autoencoder pretraining step, embedding the dataset into a lower dimension space.

First we build the autoencoder structure and train it in the adversarial setting. Then we rebuild a neural network and train it on the latent representation as supplied by the autoencoder.

I started with "the simplest possible autoencoder", as defined in the Keras documentation. We'll also add the additional activity regularization, as described in the article, to improve over the basic implementation. Slight changes were made to the architecture for clarity, and the final structure is shown in the image below.

```
input_size = X_one_hot.shape[1]
hidden_dim = input_size // 2
latent_dim = 8

# Encoder
encoder_input_layer = Input(shape=(input_size, ))
encoder_hidden_layer = Dense(hidden_dim, activation='sigmoid', activity_regularizer=regularizers.l1(10e-5))(encoder_input_layer)
encoder_output_layer = Dense(latent_dim, activation='tanh')(encoder_hidden_layer)
encoder = Model(inputs=encoder_input_layer, outputs=encoder_output_layer)

# Decoder
decoder_input_layer = Input(shape=(latent_dim,))
decoder_hidden_layer = Dense(hidden_dim, activation="tanh")(decoder_input_layer)
decoder_output_layer = Dense(input_size, activation="sigmoid")(decoder_hidden_layer)
decoder = Model(inputs=decoder_input_layer, outputs=decoder_output_layer)

# Full Architecture
autoencoder_input = Input(shape=(input_size, ))
encoded = encoder(autoencoder_input)
decoded = decoder(encoded)
ae_model = Model(inputs=autoencoder_input, outputs=decoded)
```

Figure 8: The code definition of the autoencoder architecture used for dimensionality reduction.

The latent dimension of 8 was chosen after experimentation with several latent dimension sizes. Ultimately there was not a very large difference, but 8 seemed a good middle ground. The hidden dimension size was chosen similar to the previous NN architecture, halving the size of the input dimension. During training a custom evaluation function was added that measured the reconstruction loss of the autoencoder. The image below shows the training process for 400 epochs, which resulted in a reconstruction accuracy of 97%.

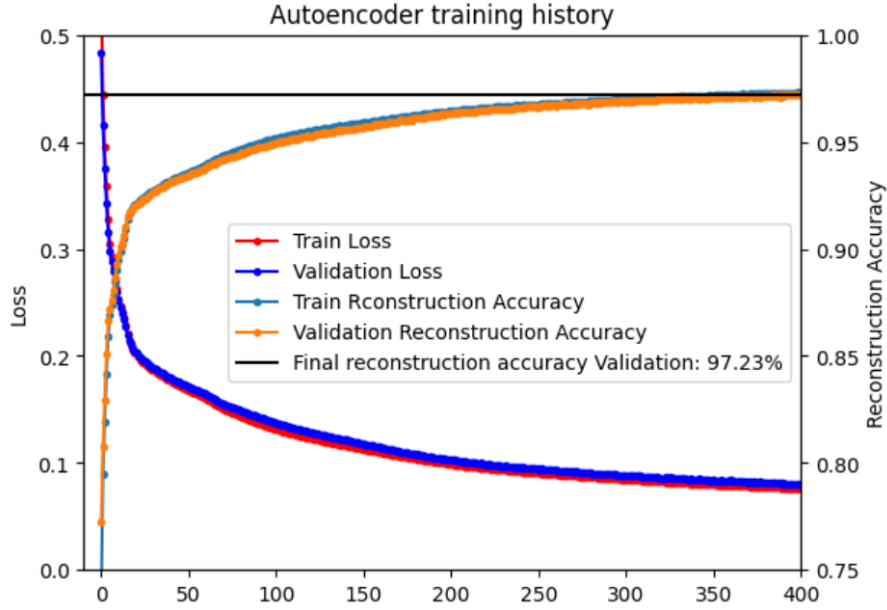


Figure 9: Autoencoder training history. After some experimentation, the amount of epochs was chosen as 400. This resulted in a 97% reconstruction accuracy on the validation set for the chosen architecture.

One important caveat is that the reconstruction error is not constant over all features. There are a few features that prove hard to capture in a latent space representation and have a reconstruction error of $> 7\%$. These features are:

- Having @-symbol
- Age of domain
- PageRank
- Google index
- Statistical report

These features are all strong predictors of phishing, as described by the authors of the dataset, and they should not suffer from reconstruction inaccuracies. Therefore, in the final architecture, they will be excluded from the dimensionality reduction stage and concatenated to the classifier input afterwards.

3.4.1 Building a classifier model on top

Now that we have a significantly reduced dimensionality for classification we can build a simple classifier network. For this we mimic the initial NN architecture with a single hidden layer. But since the input dimensionality is now only 8, we can get away with a much smaller model with only 125 parameters. Below the training history of the classifier is shown.

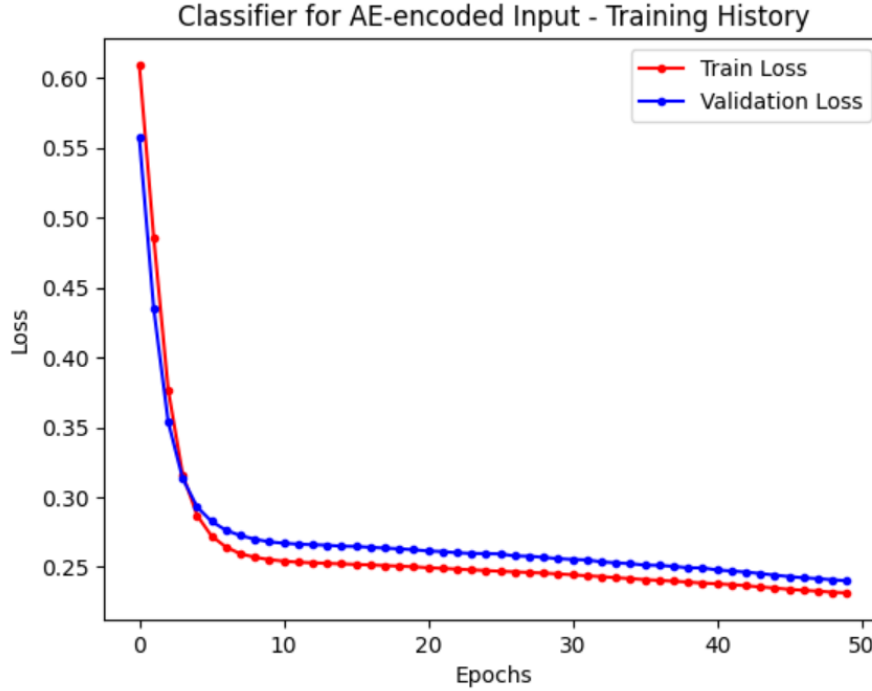


Figure 10: Training history for the autoencoder reduce classifier. Due to the small size of the model, it converges very quickly.

One major advantage is that this model converges much quicker than the previous model over the entire dataset. After only 5-10 epochs, the model is essentially done training and only minor improvements are seen. This is very useful when iterating over hyperparameters since experimentation is very quick. A disadvantage is that the accuracy is significantly lower, yielding only a 90% accuracy on the validation set. Since we are already aware of some issues in the latent representation of the data, for now that is not an issue. But when the complete architecture comes together we'll have to consider it. Once we have a trustworthy latent representation generator (the autoencoder in this case) we can quickly iterate over a subset of the data and since the autoencoder is an extremely flexible architecture, this gives plenty of room to iterate on the latent representation generation too.

For example, it is possible to prepare several iterations of the autoencoder, which are already trained on providing a latent representation. And match them to several classifier architectures. These classifier architectures will naturally be small since they only work on a subset of the data. Therefore they can quickly be tested and improved.

3.5 Interpretation

Before we bring everything together, let's inspect the latent representation a bit more. As mentioned previously, a trustworthy latent representation generator is very important for classification performance. Additionally, a major issue with latent space representations is that they are generally not very interpretable. Luckily there are several techniques we can use to make what is happening more insightful. One of these methods we already used in looking at the reconstruction error in detail. Some others that we will use here are:

- Latent Sensitivity → Changing one of the input features and looking at the change in latent space representation.
- Latent space visualisation/clustering → Visualising the latent space in lower dimensional subspace and looking at class separation.
- Latent unit activation statistics → A measurement of the decision boundary in the latent space. For each latent dimension we measure how active it is dependent on the class.

For each of the interpretation metrics, they are explained in the following sections for the basic data. Afterwards when bringing it all together in the complete architecture the metrics are recalculated and a comparison is drawn.

3.5.1 Latent Sensitivity

With latent sensitivity we perturbate the input data slightly to see how the latent space representation changes. Since all of the input variables are binary digits, we can simply flip a single feature value from 0 to 1 or vice versa while keeping the rest of the data vector fixed. Doing this for all features and plotting it as a heatmap results in the following figure.

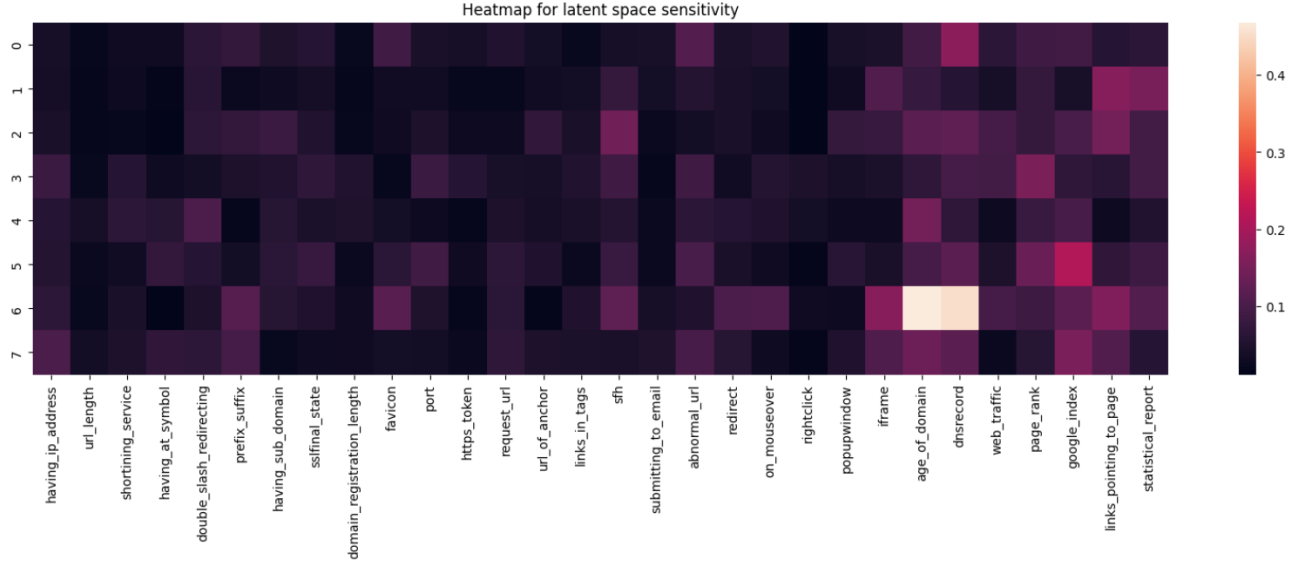


Figure 11: Heatmap of the latent space sensitivity for combination of latent feature and input feature.

This shows that most activations are not too strong on their own. The effect on latent feature 6 is largest from domain age and dns record at around 0.4. This is a good thing since the latent space is not dominated by a select subset of features.

3.5.2 Latent space visualisation

Latent space visualisation is a method to determine if the latent space provides good separation. By projecting the latent space onto a 2-dimensional subspace, it can be very insightful to see how the latent space encodes the different classes. This was done for three different projections methods. The first, a simple PCA model that only projects linearly. The other two were t-SNE and UMAP, both nonlinear models. The latent space might very clearly separate the classes in high dimensions, but not when projected directly onto 2 subdimensions. Therefore non linear models were also checked to see if a higher dimensional embedded manifold could be a better target for projection. The result is shown in the image below.

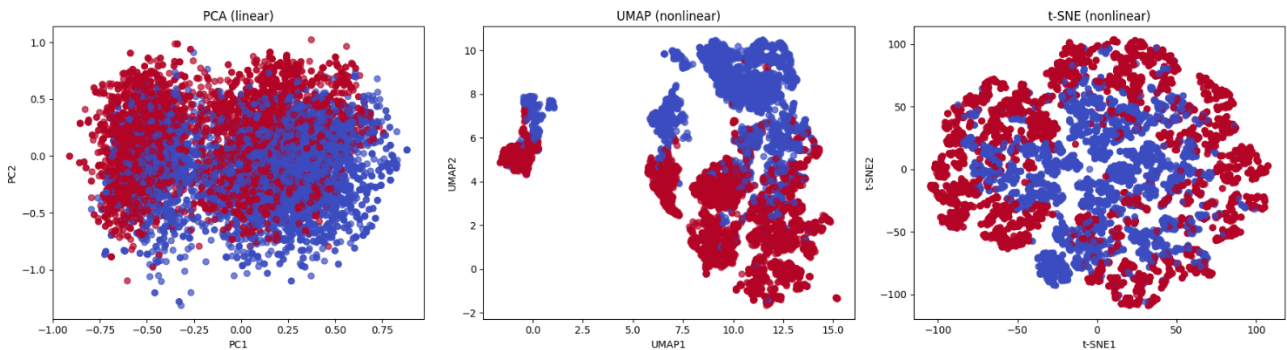


Figure 12: A comparison between three different projection models for latent space visualisation.

The linear model is reasonably disambiguating the two classes, with two regions that have majority density for one of either class. The t-SNE model does a bit better by showing one class on the inside and another on

the outside. The UMAP projection is clearly superior, showing obvious separation between the two classes. Interestingly, there seem to be two regions of density that are projected onto different sections of the subspace. But in general, UMAP shows that it is possible to separate the two classes in the latent space, leading to more evidence that the latent embedding is a proper dimensionality reduced form of the input data.

3.5.3 Latent unit activation statistics

Finally, the latent unit activation statistics were explored. Latent unit activation statistics measure how active each of the latent dimensions are for each of the input classes and how those distributions relate to each other. This analysis gives insight in what the latent dimensions mean and whether they encode general information, specific higher-level concepts or combinations directly related to the output classes. Below the average activation per class is shown for each of the latent dimensions. This shows that there is a clear trend towards a specific class in most of the responses of the latent dimensions.

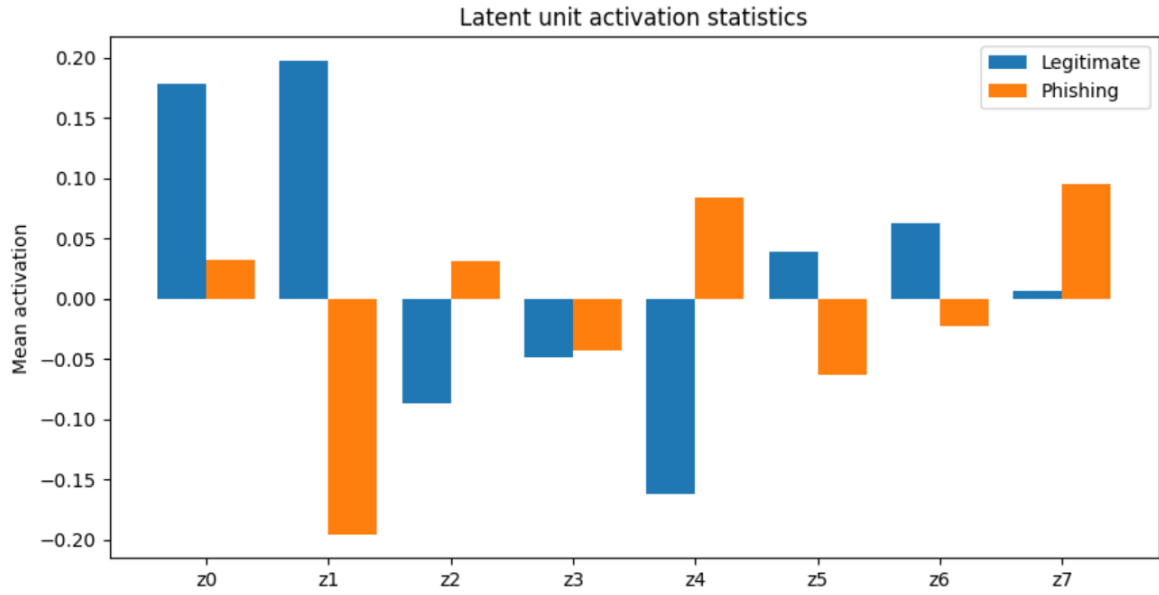


Figure 13: Average activation of the latent space divided by class.

Another way we can visualise this is by plotting the difference between the activations. This shows a clear trend in how the latent space response differs between classes.

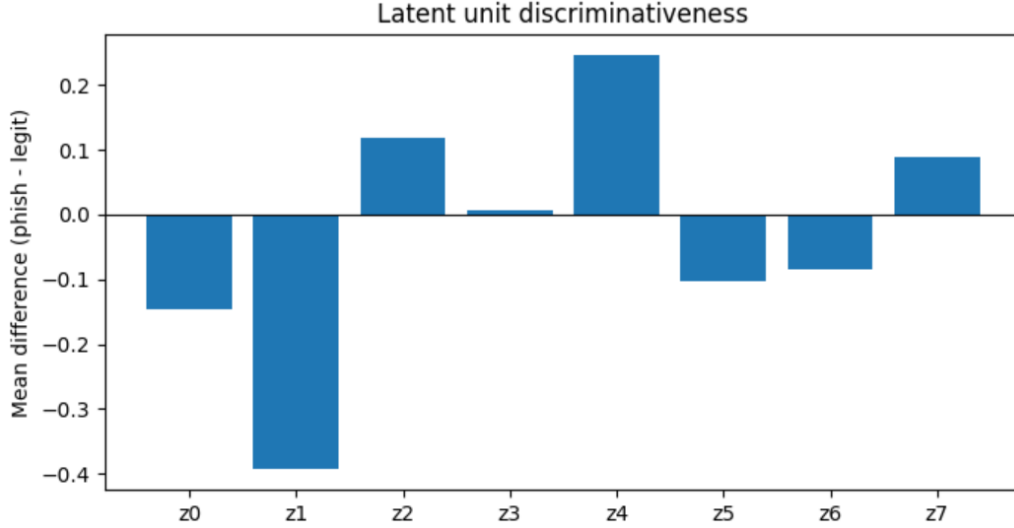


Figure 14: Difference of the average response of the latent space between classes. A strong positive response indicates that the latent unit responds stronger to phishing websites.

From this analysis we can see that the discriminative power of the latent space representation is very good. Just one dimension does not provide significant discrimination between classes. This suggests that the autoencoder actually learns a meaningful distinction between phishing and non phishing sites. Some latent dimensions seem to encode specifically for a class (where one of the responses is large and the other small) and some seem to encode for differences between classes (where both responses are opposite and similarly sized). For example, dimension z_0 encodes for legitimate sites whereas dimension z_7 encodes for phishing sites. Dimension z_1 encodes strongly for a difference between phishing- and non-phishing sites.

Having a single dimension that provides no meaningful distinction might indicate that the size of the latent space is slightly to large. Possibly, a latent dimension of size 7 could also suffice with similar performance. However, this is also greatly dependent on the autoencoder hyperparameters and overall structure.

An interesting questions that arises here is how to interpret the results. We can see from the latent activation that the average response is stronger for non-phishing websites. It can be an interesting followup to research how shaping of the latent space can result in better classifier results later in the chain. A latent space that responds stronger to positives might increase the false positive rate later on, while a latent space that responds stronger to negatives could increase the false negative rate. Depending on the problem and the consequences, latent space modeling might be a good method of increasing performance on this specific axis.

3.6 Updating the model

Now we have a good baseline and a strong insight in how modeling influences the results we can design an informed model for this use case. The following choices will be made, based on the previous analysis:

- A few columns that were not properly encoded as latent space representations (large reconstruction error) will be removed from the autoencoder dimensionality reduction step. They will be appended separately to the classifier input.
- A latent dimension size of 7 will be used. This is because there was already a bit of redundancy found with the latent space activation statistics. Removing columns with large reconstruction errors is expected to require a smaller latent space representation.
- Due to the change of the latent space dimensionality and the appending of unencoded columns, the input to the classifier layer will be different. Due to this the hidden layer shape will also be changed to reflect this. The input to the classifier will be $d_{unencoded} + d_{latent} = d_{classifier_input}$, which in this case will be $5 + 7 = 12$, the hidden layer size will be $\frac{d_{classifier_input}}{2} = 6$.
- The autoencoder will still be trained separately, but an additional full hybrid architecture is also suggested that can be trained as a full system. An advantage is that this architecture can be deployed easily, but the cost for that is increased training time an reduced modularity.

First, the models were rebuilt and trained. Here they showed similar behavior to the previous iteration.

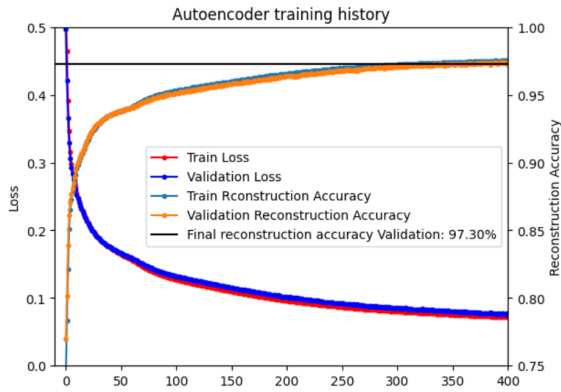


Figure 15: Train history for the revised autoencoder. The model was trained for 400 epochs and shows good reconstruction quality.

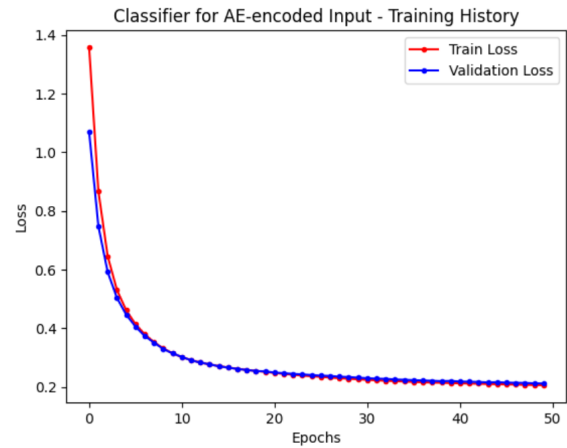


Figure 16: Train history for the revised classifier. The model was trained for 50 epochs, getting the 12-dimensional input vector made from combined encoded and non-encoded inputs.

After training this in separate sections (autoencoder and classifier), the models were combined. This was done with the following code:

```
ENC_SIZE = ENCODER_INPUT_SIZE
BYPASS_SIZE = len(exclude_from_dim_red_cols)
FULL_INPUT_SIZE = ENC_SIZE + BYPASS_SIZE

# To select the correct columns in the model we need to convert them to indices
ALL_COLUMNS = X_one_hot.columns
BYPASS_COLS = exclude_from_dim_red_cols
ENCODE_COLS = [c for c in ALL_COLUMNS if c not in BYPASS_COLS]
ENCODE_IDX = [ALL_COLUMNS.get_loc(c) for c in ENCODE_COLS]
BYPASS_IDX = [ALL_COLUMNS.get_loc(c) for c in BYPASS_COLS]

full_input = Input(shape=(FULL_INPUT_SIZE,))
encoded_part = Lambda(lambda x: tf.gather(x, ENCODE_IDX, axis=1), output_shape=(ENC_SIZE,))(full_input)
bypass_part = Lambda(lambda x: tf.gather(x, BYPASS_IDX, axis=1), output_shape=(BYPASS_SIZE,))(full_input)
latent = encoder(encoded_part)
classifier_input = Concatenate()([latent, bypass_part])
classifier_output = classifier_model(classifier_input)
full_model = Model(inputs=full_input, outputs=classifier_output)

encoder.trainable = False
classifier_model.trainable = False
```

Figure 17: Code for combining the encoder portion and the classifier portion.

We simply connect the output of the encoder to the input of the classifier and add a preprocessing layer that split the input data into a section to be encoded using the autoencoder and a section to bypass the autoencoder. Finally, a classification report was generated for the entire architecture.

	precision	recall	f1-score	support
0	0.92	0.88	0.90	4898
1	0.91	0.94	0.92	6157
accuracy			0.91	11055
macro avg	0.91	0.91	0.91	11055
weighted avg	0.91	0.91	0.91	11055

Figure 18: Classification report for the combined architecture (separately trained)

Simply combining the model only gives an accuracy of 91%, again very similar to the previous iteration.

3.6.1 Hybrid architecture

Finally, a hybrid architecture was used. In this architecture the model was immediately combined before compilation and trained as a single unit. For this the autoencoder got its own output head which could be evaluated with a loss function, together with the classifier output. Additionally, layers were added for input consolidation, and concatenation. The full architecture is shown below

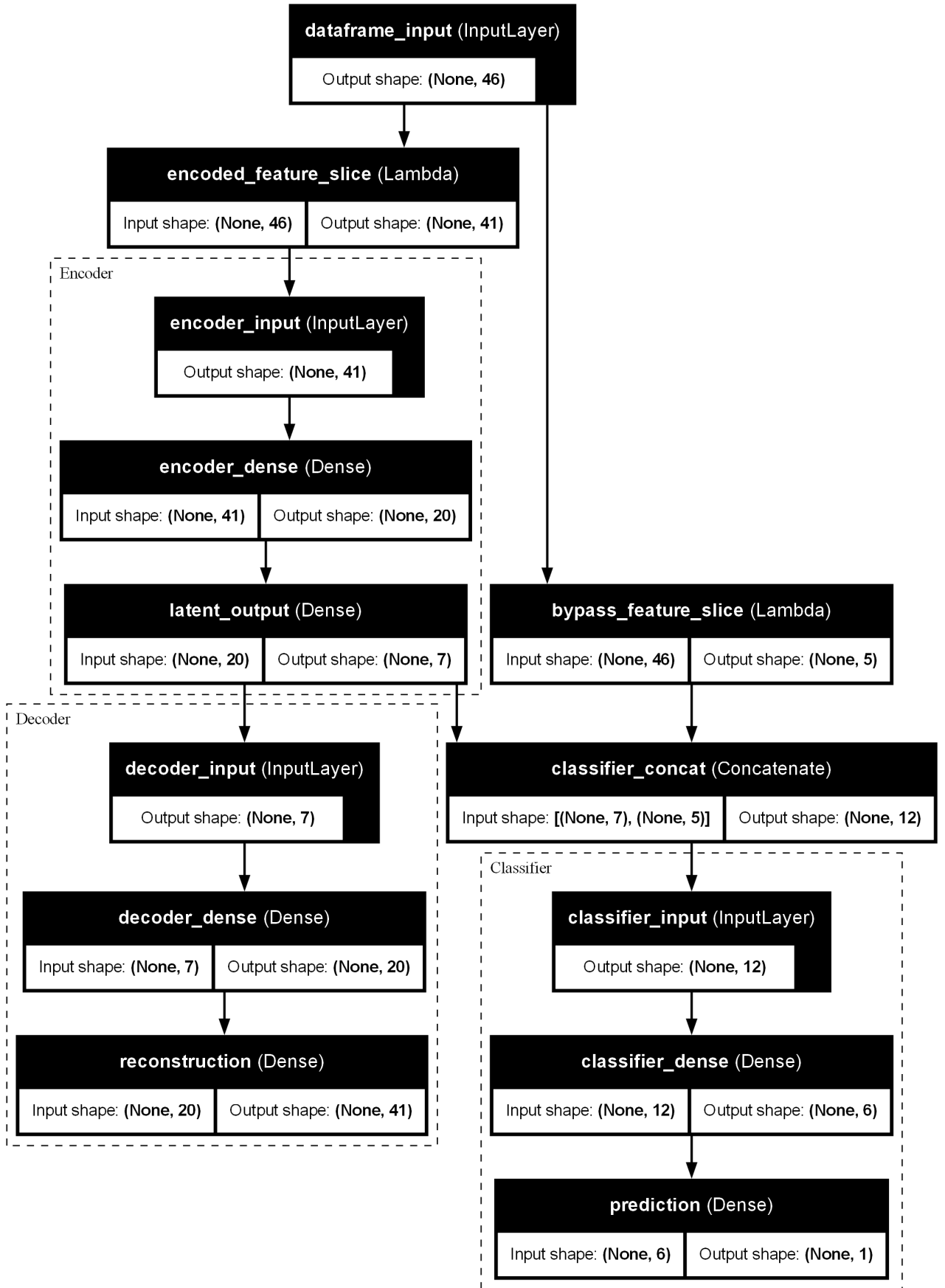


Figure 19: Diagram of the complete model structure for the hybrid architecture.

The complete architecture was then trained as a single unit, learning the latent representation at the same time as the classifier was trained. Therefore, due to the combined evaluation of the loss function, the autoencoder learned a latent space representation that served the classifier much better. This lead to a classification accuracy of around 96%.

	precision	recall	f1-score	support
0	0.94	0.98	0.96	4898
1	0.98	0.95	0.97	6157
accuracy			0.96	11055
macro avg	0.96	0.96	0.96	11055
weighted avg	0.96	0.96	0.96	11055

Figure 20: Classification report for the hybrid architecture. (trained together)

This is similar in performance to the classical ML model baseline that was taken earlier.

3.6.2 Hybrid Interpretability

The same 3 latent space interpretability methods were used for evaluating the hybrid model. They are shown below, where we can see that the discriminativeness and the latent representation still shows the same beneficial properties for the hybrid architecture.

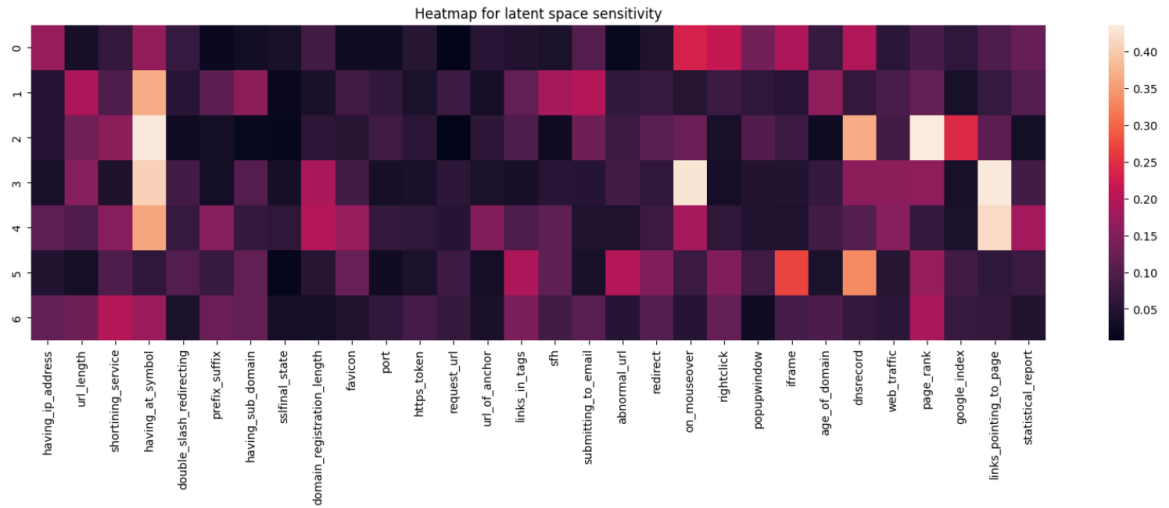


Figure 21: Heatmap visualisation of latent space sensitivity. The activity is a bit stronger for some of the input features, but still there is no strong preference either way. This still shows a good distribution.

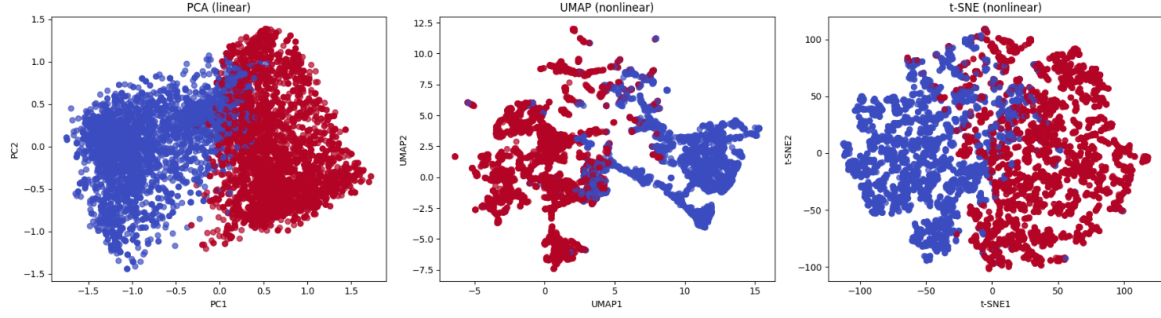


Figure 22: Projection of the latent space on two dimensions, colored by class. This shows if the latent space is properly discriminating phishing from non-phishing sites. In this model, the decision boundary is much more linear than the previous case. This makes sense since the latent representation is built together with the classifier. Therefore it is more likely that the latent space learns a representation that is easier to separate.

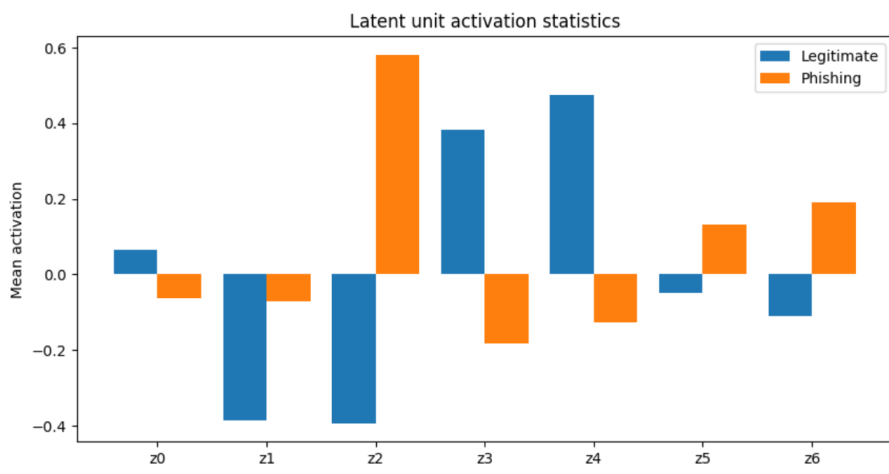


Figure 23: The latent activation statistics for the hybrid architecture encoder. This again shows that most of the latent dimensions strongly encode for discriminativeness between classes or for one of the two classes.

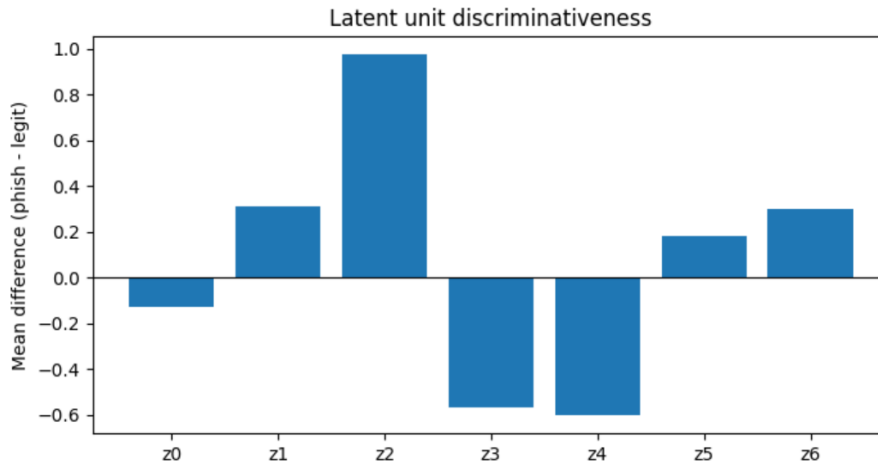


Figure 24: Explicit discriminativeness between classes for each latent unit. Since only a few latent dimensions clearly discriminate between classes, this might mean that the latent dimension can be reduced even more.

4 Summary and Reflection

In this article, a hybrid neural network architecture was explored for detecting phishing websites. The model was compared to several classical models (Logistic Regression, SVC and Random Forest). The first iteration was a simple classifier that directly predicted on the input data. The second iteration added also trained an autoencoder separately as a preprocessing step. The third iteration took insights from the second iteration (latent space size, excluding variables from the encoding step) and retrained the architecture separately (first the encoder and then the classifier on top). Finally a hybrid model was trained as a combined unit, training the latent space embedding together with the classifier layer. This model performed similarly to the classical case with an accuracy of 96%.

The complexity of the model is not directly in proportion to an increase in performance in this specific case. But the architecture is very flexible and can easily be adapted to more complex problems with lower degrees of linearity. This project has proven that the architecture works conceptually.

4.1 Next Steps

A good next step would be to convert the complete architecture to a general model that takes in hyperparameters for layer sizes, network depth, activation functions and included/excluded data. Then to apply it to different datasets and compare with a classical baseline. More exploration of the latent space is also an interesting area to see if a meaningful representation can be achieved. Since the model uses an autoencoder, it can also be used generatively. Something that is less relevant for this dataset, but it should be easy to find applications for this property. If this would be a desired outcome, it might be better to use a variational autoencoder to get a more meaningful and interpretable representation in the latent space.